Strength Training and Performance Tracking System

Shreeya Upendra Gadgil

Haris Asir Hasan

Milan Limbu

Aliza Shrestha

Charles Ssekitto

University of the Cumberlands

MSCS 532 M70 Algorithms and Data Structures

Dr. Michael Solomon

November 6, 2026

Strength Training and Performance Tracking System

Application Context and Scope

This Python prototype lets multiple users record completed strength-training workouts, retrieve a workout by ID, search their own workouts by UTC date range, and find their heaviest completed lift per exercise. Each workout has a user-scoped ID, exercise ID, timestamp, sets (weight in kilograms and positive repetition count). A personal record (PR) is the greatest valid set weight; equal weights retain the first logged occurrence. Users and exercises are preloaded. Authentication, editing, deletion, concurrent writes, persistent storage, running, calories, and endurance metrics are excluded.

Selected Data Structures and Alternatives

Exact retrieval and user isolation: A top-level hash table maps user IDs to user records; each user has a workout-ID hash table and exercise-ID PR hash table. Expected O(1) exact lookup is preferable to AVL-tree O(log n) lookup or linear array scanning. Hashing provides no inherent date ordering, and collision behavior depends on the implementation (Carter & Wegman, 1979).

Date-range retrieval: Each user has an AVL tree keyed by (UTC timestamp, workout ID), whose nodes reference workouts. Balanced height supports worst-case O(log n) insertion and O(log n + k) reporting of k matching workouts, including backdated entries (Adel'son-Vel'skii & Landis, 1962). A sorted array has O(n) middle insertion; an ordinary BST can skew to O(n); a linked list requires linear date search. Tree variant performance also depends on access patterns (Pfaff, 2004).

Workout sets and PRs: A dynamic array supports amortized O(1) append and efficient sequential traversal. Linked lists incur pointer overhead and slow indexed access; trees add needless ordering. A per-exercise cached maximum provides constant-time comparison per incoming set and expected O(1) PR retrieval. A max-heap would add O(log n) insertion work without improving maximum-only queries; it becomes more useful for repeated maximum removal or top-ranked retrieval.

Complexity and Space Tradeoffs

Let n be the selected user's workout count, s the number of sets in a new workout, k the number returned in a date query, U the number of users, W the total workouts, and S the total sets. Adding a workout costs expected amortized O(s + log(n + 1)); its AVL insertion is worst-case O(log(n + 1)). Getting a workout by ID or a PR is expected O(1); a date-range query is expected O(1) for user lookup plus worst-case O(log(n + 1) + k) for the AVL traversal. Total storage is O(U + W + S), assuming bounded-length IDs and names. Index references, not duplicated workout objects, limit overhead. Individual hash-table resizes or pathological collisions may be more expensive.

Implementation Handoff

Operation contracts: add_workout(user_id, workout_id, exercise_id, timestamp, sets) returns the accepted workout; get_workout(user_id, workout_id) returns a workout or no result; search_dates(user_id, start_utc, end_utc) returns an ordered list, including both boundaries; get_personal_record(user_id, exercise_id) returns (workout ID, set index, weight, repetitions, timestamp), or no result. Workout IDs are unique within a user; the pair (workout ID, zero-based set index) identifies a set.

Validation and testing: reject unknown user/exercise, duplicate workout ID, empty sets, nonpositive or nonfinite weights, nonpositive or noninteger repetitions, naive/invalid timestamps, and reversed date bounds. Convert offset-aware timestamps to UTC; equal timestamps order by workout ID. An empty valid range returns []. Successful updates synchronize all indexes; on recoverable partial failure, rebuild user indexes from accepted workouts. Crash recovery is excluded.

References

Adel'son-Vel'skii, G. M., & Landis, E. M. (1962). An algorithm for organization of information. Doklady Akademii Nauk SSSR, 146(2), 263–266. https://www.mathnet.ru/eng/dan26964

Carter, J. L., & Wegman, M. N. (1979). Universal classes of hash functions. Journal of Computer and System Sciences, 18(2), 143–154. https://doi.org/10.1016/0022-0000(79)90044-8

Pfaff, B. (2004). Performance analysis of BSTs in system software. In Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems (pp. 410–411). Association for Computing Machinery. https://doi.org/10.1145/1005686.1005742

AI Usage Appendix

Significant Prompts

1. "How would we tackle scalability across multiple users?"
2. "What alternative data structures can improve the system performance?"
3. "Challenge the system assumptions and identify its incorrect complexity claims"
4. "Compare it with linked lists, heaps, and balanced search trees"